6-2 · Docker 기반 Kafka 클러스터 · 이론 1H

KRaft 필수 설정과 리스너 2계층 구조

접속은 되는데 메시지가 안 오간다 — 원인은 리스너

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① listeners 와 advertised.listeners 의 차이를 설명한다

② 컨테이너 안과 밖에서 닿는 주소가 왜 다른지 안다

③ 단일 브로커에서 내부 토픽 복제 수를 1로 주는 이유를 안다

④ 컨슈머 그룹의 파티션 분배와 LAG 의 뜻을 말할 수 있다

다음 2.5H 실습에서 이 설정을 compose 로 직접 쓰고 두 경로로 접속해 확인합니다

1. 두 번 접속

클라이언트는 두 번 접속한다 — 대표번호, 그리고 직통번호

(1) 첫 접속	(2) 안내	(3) 재접속
bootstrap-server 로 아무 브로커에나	「나에게 다시 붙으려면 이 주소로」	안내받은 주소로 실제 송수신

(2)의 주소가 advertised.listeners — Kafka 에서 가장 흔한 장애가 여기서 난다

(2)에서 알려 준 주소가 클라이언트 쪽에서 닿지 않으면 (1)은 성공하고 (3)에서 조용히 끊긴다

1. 두 번 접속

listeners 와 advertised.listeners — 둘은 다를 수 있다

listeners — 실제로 여는 포트

전화기를 놓는 자리

- 브로커가 실제로 열어 두는 포트
- 예: :29092 를 연다
- 열려 있어도 안내가 틀리면 소용없다

VS

advertised.listeners — 안내 주소

안내하는 직통번호

- 클라이언트에 「이 주소로 다시 접속하세요」
- 예: kafka:29092 라고 안내
- 클라이언트 입장에서 닿는 주소여야 한다

2. 리스너 2계층

왜 리스너가 둘 이상인가 — 누가 붙느냐에 따라 주소가 다르다

접속하는 쪽	닿는 주소	리스너 이름
같은 compose 안의 다른 컨테이너	kafka:29092	PLAINTEXT
내 컴퓨터(호스트)의 프로그램	localhost:9092	EXTERNAL
브로커끼리(메타데이터 합의)	kafka:29093	CONTROLLER

1-5·4-1 에서 본 「밖에서는 localhost, 안에서는 서비스 이름」이 설정으로 드러나는 자리

리스너를 하나만 두면 둘 중 하나가 반드시 깨진다 — 그래서 PLAINTEXT·EXTERNAL 둘 다 선언한다

KRaft — ZooKeeper 없이 브로커가 컨트롤러를 겸한다

설정(KAFKA_ 뒤)	뜻
NODE_ID	이 노드의 번호 — 클러스터에서 고유
PROCESS_ROLES	broker,controller — 두 역할을 겸함
CONTROLLER_QUORUM_VOTERS	1@kafka:29093 — 컨트롤러 후보 목록
CONTROLLER_LISTENER_NAMES	컨트롤러 통신에 쓸 리스너 이름
INTER_BROKER_LISTENER_NAME	브로커끼리 쓸 리스너 이름
*_REPLICATION_FACTOR: 1	내부 토픽 복제 수 — 단일 브로커라 1

단일 브로커에서는 복제 수 설정 3개를 반드시 1로 — 내부 토픽 생성이 실패하면 컨슈머 그룹이 동작하지 않는다

4. 컨슈머 그룹

같은 그룹의 컨슈머가 파티션을 나눠 갖는다

상황	무슨 일이
파티션 3, 컨슈머 1	한 컨슈머가 3개 다 맡음
파티션 3, 컨슈머 2	2개 + 1개로 나눔
파티션 3, 컨슈머 3	하나씩
파티션 3, 컨슈머 4	한 명은 논다 — 파티션보다 많으면 낭비
컨슈머 하나가 죽음	리밸런싱 — 남은 컨슈머가 넘겨받음
다른 그룹의 컨슈머	독립적으로 전부 읽음

병렬로 읽을 수 있는 상한 = 파티션 수 — 5주차 Spark 의 「파티션 수가 병렬의 상한」과 같은 이야기

5. LAG

LAG — 얼마나 밀렸는가

1. **LOG-END-OFFSET**
   그 파티션에 쌓인 마지막 위치 — Producer 가 만든 것
2. **CURRENT-OFFSET**
   컨슈머가 읽은 위치
3. **LAG = LOG-END-OFFSET − CURRENT-OFFSET**
   0 이면 따라잡은 것, 계속 커지면 소비가 생산을 못 따라가는 것

운영에서 가장 먼저 보는 지표

LAG 0

= 따라잡음

6. 보존 기간

retention — 며칠치를 되감아 읽을 수 있어야 하는가

1. **retention.ms**
   토픽의 메시지 보존 기간 — 지나면 오래된 세그먼트부터 삭제, 기본 7일
2. **retention.bytes**
   용량 기준 보존 — 둘 중 먼저 걸리는 쪽이 적용된다
3. **지나면 못 읽는다**
   보존 기간이 지나면 --from-beginning 으로도 못 읽는다
4. **정하는 기준**
   「며칠치를 되감아 읽을 수 있어야 하는가」— 재처리가 잦으면 길게

정리

이 1시간에서 꼭 가져갈 것

1. 안내받은 주소로 다시 붙는다 — advertised.listeners 가 틀리면 조용히 깨진다
2. 컨테이너 안과 밖은 닿는 주소가 달라 리스너를 둘 이상 둔다
3. 단일 브로커에서는 내부 토픽 복제 수를 전부 1로 준다
4. 같은 그룹의 컨슈머가 파티션을 나눠 갖고, 파티션 수가 병렬의 상한
5. LAG 은 「얼마나 밀렸는가」— 운영에서 가장 먼저 보는 지표

6-2 · 이론 1H 마무리

다음은 토픽 관리·컨슈머 그룹 실습 (2.5H)

compose 로 KRaft 브로커를 띄우고, 두 경로로 붙고, 컨슈머를 끄고 늘리며 LAG 과 리밸런싱을 눈으로 확인합니다

- ✓ **docker compose 로 KRaft 브로커 기동**
- ✓ **kafka:29092 · localhost:9092 두 경로 접속 확인**
- ✓ **orders 토픽 생성 · retention.ms 변경·복원**
- ✓ **컨슈머 그룹 LAG · 분배 · 리밸런싱 관찰**

준비할 것

docker rm -fv kafka · week6 폴더